

For any other rendering engine, you need to specify to Express which engine to use.

Your Express project

Static HTML files

First, let us learn to host a website with static HTML files. Express makes it very simple to do so. Just copy the set of HTML/CSS/JavaScript files under the directory 'public'. Let us assume that you copied all these static files under a directory 'sample'. To view those pages, point your browser to `http://localhost:3000/sample/`. If there is a file `index.html` in that directory, it gets opened in the browser. HTML file-hosting is possible because of one line in the `app.js` file generated by Express in Template. We will look at the code in the next section.

Photoviewer

Now, let's host a photoviewer server that displays all pictures on the server (specific directory) as thumbnails and also displays the full images when clicked. This is possible with a few code changes in the already generated template of Express.

File/Directory	Contents
<code>app.js</code>	This is like the <code>main()</code> function. The execution starts from this file.
<code>package.json</code>	This has instructions such as dependencies, on how to run an app, etc.
Images	All images required for the project are stored here.
<code>node_modules</code>	Dependent node packages are stored locally to the app.
public	Stores all the static Web pages.
routes	Conditional and flexible routing of requests.
views	Contains dynamically generated content.

Replace the contents of the `photoviewer/app.js` file with the code given below. After the `app.js` file is saved, remember to stop and start the node server (run 'node app' at the command prompt).

```

1  /**
2   * Module dependencies.
3   */
4
5  var express = require('express');
6  var routes = require('./routes');
7  var http = require('http');
8  var path = require('path');
9
10 var app = express();
11
```

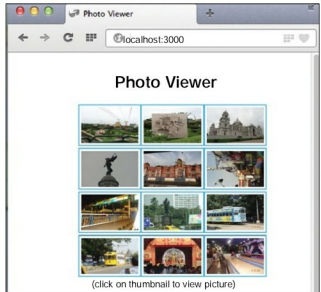


Figure 3: Photoviewer

```

12 app.set('port', process.env.PORT || 3000);
13 app.set('views', __dirname + '/views');
14 app.set('view engine', 'jade');
15 app.use(app.router);
16 app.use(express.static(path.join(__dirname, 'public')));
17
18 app.get('/', routes.index);
19
20 app.get('/images/mypics/:file', function(req, res) {
21     res.sendFile(__dirname + '/images/mypics/' +
22                 req.params.file);
23 });
24 http.createServer(app).listen(app.get('port'),
25                               function(){
26 console.log('Photo viewer started at
http://localhost:' + app.get('port'));
26 });

```

Here is the explanation for the code given above.

Lines 5 - 8: Initialises objects from their respective node modules of Express, `routes`, `http` and `path`. Express is the main object that allows the setting of application specific variables and also invoking of middleware. `Routes` allows the requests from clients to be processed by various callback functions. `http` is the object that helps in the creation of the server. The `path` package has a set of utilities for file and path handling.

Line 10: `app` is the object used throughout the application, for setting the attributes and using middleware functionality.

Lines 12 - 14: Sets the port where the server should listen for the client to connect. For HTTP, the default is 80. Express uses 3000 as the default port. By default, the 'views' directory contains dynamic content.